

Nama : Evia Vinata

NIM : 245150607111009

Pengembangan Aplikasi Peranti Bergerak

Percobaan

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Pra-Percobaan

Pada percobaan ini dibuat dua project terpisah dengan hasil akhir yang sama. Teks awal adalah "Belum diklik". Setelah tombol "Ubah Teks" ditekan, teks berubah menjadi "Tombol sudah diklik". Perbedaan yang diamati bukan hasil akhirnya, melainkan cara program memperbarui UI.

Project A - XML/View (Imperatif)

1. Buat project Android Studio baru menggunakan template Empty Views Activity. Beri nama project ImperativeXmlApp dan pilih Kotlin.
2. Buka res/layout/activity_main.xml, kemudian gunakan kode berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Belum diklik"
        android:textSize="24sp"
        android:layout_marginBottom="16dp" />

    <Button
        android:id="@+id/btnChange"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ubah Teks" />
</LinearLayout>
```

3. Buka MainActivity.kt. Sesuaikan package dengan package project, kemudian gunakan kode berikut:

```
package com.example.imperativexmlapp
```

```

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnChange = findViewById<Button>(R.id.btnChange)

        btnChange.setOnClickListener {
            tvMessage.text = "Tombol sudah diklik"
        }
    }
}

```

4. Jalankan aplikasi. Pastikan teks berubah setelah tombol diklik.

Project B - Jetpack Compose (Deklaratif)

1. Buat project Android Studio baru menggunakan template Empty Activity untuk Jetpack Compose. Beri nama project DeclarativeComposeApp dan pilih Kotlin.
2. Buka MainActivity.kt, pertahankan package project, kemudian gunakan fungsi composable berikut:

```

@Composable
fun ChangeTextScreen() {
    var message by remember {
        mutableStateOf("Belum diklik")
    }

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = message, fontSize = 24.sp)
        Button(onClick = {
            message = "Tombol sudah diklik"
        }) {
            Text("Ubah Teks")
        }
    }
}

```

3. Tambahkan import yang diperlukan:

```

import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button

```

```
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.sp
```

4. Pada fungsi onCreate(), ubah isi setContent agar memanggil ChangeTextScreen():

```
setContent {
    ChangeTextScreen()
}
```

5. Jalankan aplikasi. Pastikan hasil tampilannya sama dengan project XML/View.

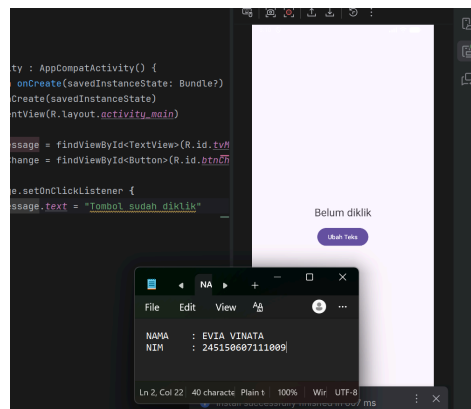
Aktivitas Percobaan

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

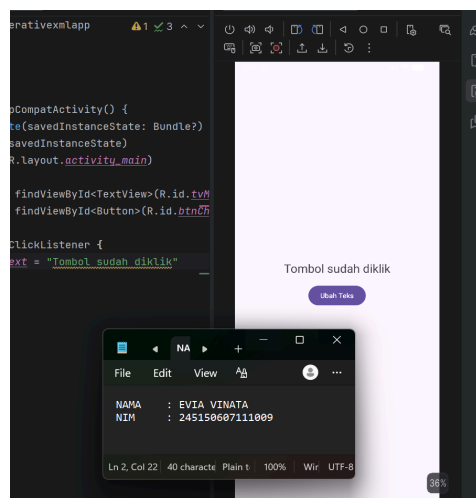
Jawab :

1. Perbandingan Imperatif

- Sebelum diklik

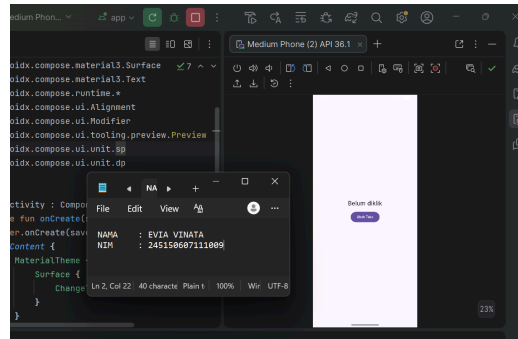


- Sesudah diklik

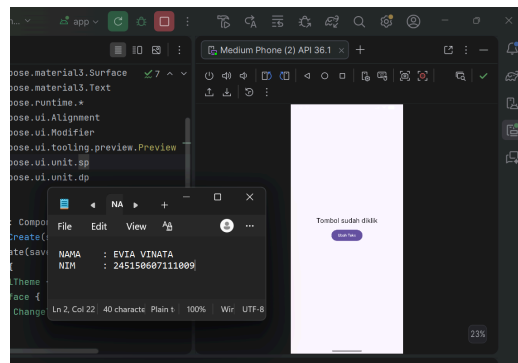


2. Perbandingan Imperatif

- Sebelum diklik



- Sesudah diklik



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawab : Objek yang diubah secara langsung adalah TextView tvMessage, melalui referensi hasil findViewById()

```
val tvMessage = findViewById<TextView>(R.id.tvMessage)
val btnChange = findViewById<Button>(R.id.btnChange)

btnChange.setOnClickListener {
    tvMessage.text = "Tombol sudah diklik"
}
```

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Jawab : Yang diubah bukan komponen UI secara langsung, melainkan state bernama message.

```
var message by remember { mutableStateOf("Belum diklik") }
...
message = "Tombol sudah diklik"
```

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Jawab : Karena Compose menganut pendekatan deklaratif: tampilan (UI) dideskripsikan sebagai fungsi dari state ($UI = f(state)$), bukan dikendalikan lewat referensi objek yang diubah manual. Composable `Text(text = message)` hanya membaca nilai state saat ini; ketika state `message` berubah, Compose secara otomatis menjalankan recomposition dan menggambar ulang `Text` dengan nilai baru. Pengembang tidak perlu (dan tidak boleh) mencari objek `Text` lalu memerintahkannya berubah secara langsung.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Jawab : Tombol diklik event `onClick` memicu program mengambil referensi View (`TextView`) lewat `findViewById()` properti `text` pada View tersebut diubah secara langsung tampilan berubah karena View itu sendiri yang dimodifikasi.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Jawab : Tombol diklik event `onClick` memicu nilai state (`message`) diubah Compose mendeteksi bahwa state yang diamati (`observable state`) telah berubah Compose menjalankan recomposition, yaitu menjalankan ulang fungsi composable yang membaca state tersebut hasil recomposition menghasilkan perubahan tampilan pada `Text`, tanpa program pernah memerintahkan objek `Text` secara langsung.

7. Apa yang terjadi jika `message` diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jawab : Teks pada layar tidak akan berubah secara visual walaupun tombol tetap bisa diklik dan nilai variabel di memori berubah. Penyebabnya, variabel biasa bukan `observable state` bagi Compose. Compose hanya mengetahui perlunya recomposition jika perubahan terjadi pada objek state seperti hasil `mutableStateOf()`. Karena Compose tidak "mengawasi" variabel biasa, tidak ada sinyal yang memicu recomposition, sehingga composable `Text` tidak pernah dijalankan ulang dan tampilan tetap menunjukkan nilai awal.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Jawab : Cara berpikir imperatif berfokus pada "bagaimana": program secara eksplisit mencari komponen UI lalu memerintahkannya berubah langkah demi langkah. Cara berpikir deklaratif berfokus pada "apa": pengembang cukup mendeskripsikan tampilan seharusnya seperti apa berdasarkan state saat ini, dan framework (Compose) yang bertanggung jawab menentukan bagian mana yang perlu digambar ulang ketika state berubah. Deklaratif membuat kode lebih ringkas dan mengurangi risiko lupa memperbarui salah satu komponen UI secara manual.

Event Handling State

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity:

`@Composable`

```
fun EventStateTest() {  
    var count = 0  
    Column(  
        verticalArrangement = Arrangement.Center,  
        horizontalAlignment = Alignment.CenterHorizontally,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        Text("Counter : $count")  
        Button(onClick = {  
            count++  
            Log.d("TEST", "Counter berubah : $count")  
        }) {  
            Text("Tambah")  
        }  
    }  
}
```

3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```
import androidx.compose.foundation.layout.Arrangement  
import androidx.compose.foundation.layout.Column  
import androidx.compose.foundation.layout.fillMaxSize  
import androidx.compose.material3.Button  
import androidx.compose.material3.Text  
import androidx.compose.runtime.Composable  
import androidx.compose.ui.Alignment  
import androidx.compose.ui.Modifier  
import androidx.compose.ui.tooling.preview.Preview
```

4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

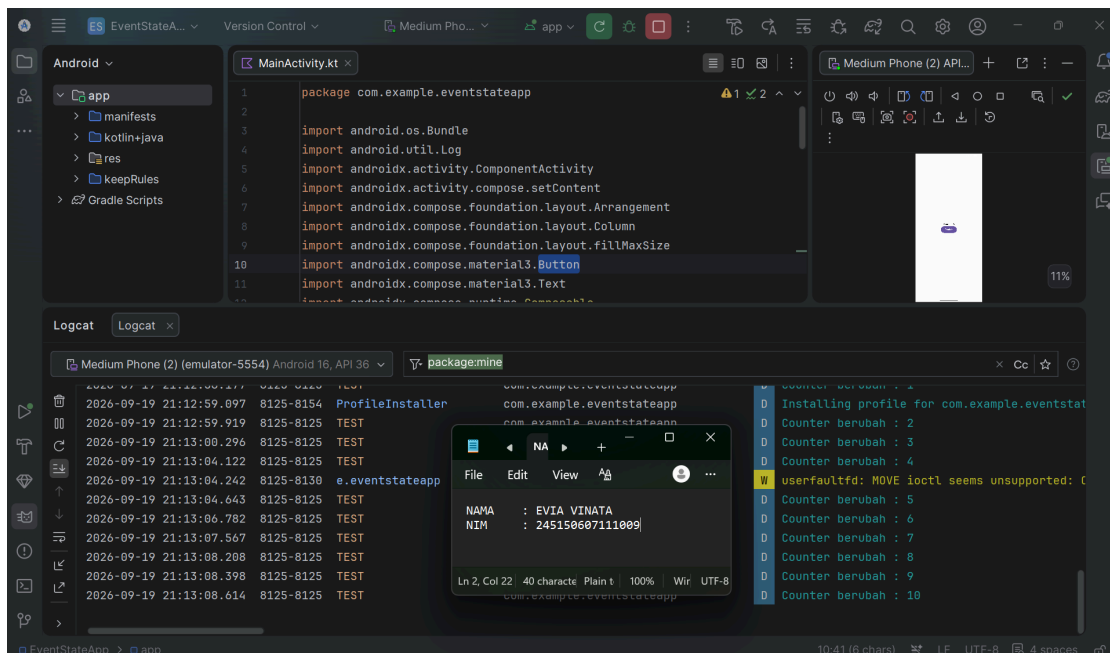
```
setContent {
    EventStateTest()
}
```

5. Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawab :



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawab : Muncul log baru setiap kali diklik, misalnya "Counter berubah : 1", "Counter berubah : 2", dst., karena baris Log.d(...) tetap dieksekusi setiap kali lambda onClick dijalankan, terlepas dari apakah UI ikut berubah atau tidak.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab : Tampilan Text("Counter : \$count") tetap menunjukkan "Counter : 0" dan tidak berubah sama sekali, walaupun nilai count di memori sebenarnya bertambah.

4. Apa yang menyebabkan hal tersebut terjadi?

Jawab : Karena count hanyalah variabel Kotlin biasa (var count = 0), bukan Compose state. Compose tidak memiliki mekanisme untuk mengamati perubahan pada variabel

biasa, sehingga tidak ada recomposition yang dipicu dan Text tidak pernah dijalankan ulang dengan nilai count yang baru.

- Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.
- Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

```
var count = 0
```

menjadi:

```
var count by remember { mutableStateOf(0) }
```

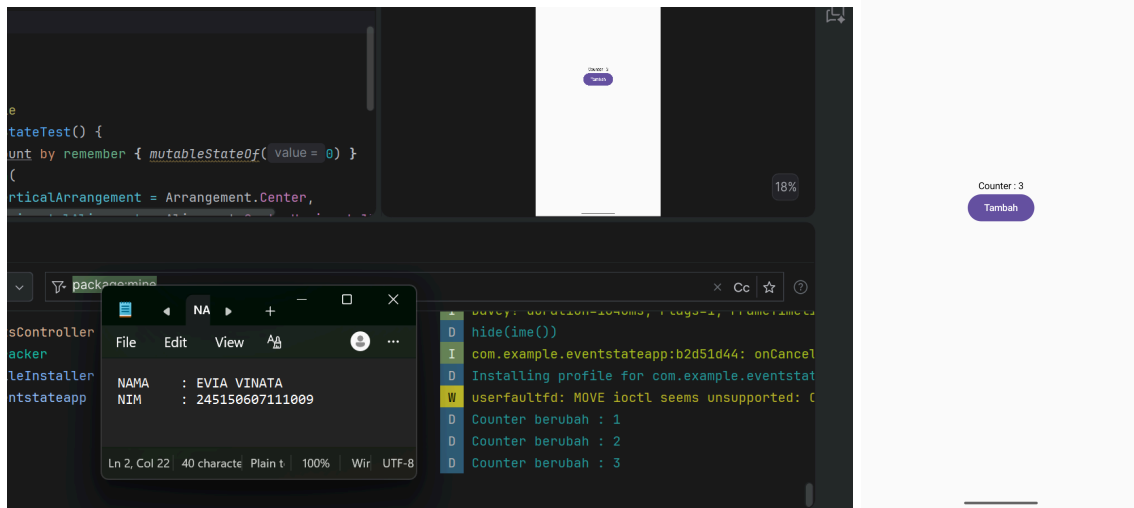
- Berikut kode lengkap EventStateTest:

@Composable

```
fun EventStateTest() {  
    var count by remember { mutableStateOf(0) }  
    Column(  
        verticalArrangement = Arrangement.Center,  
        horizontalAlignment = Alignment.CenterHorizontally,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        Text("Counter : $count")  
        Button(onClick = {  
            count++  
            Log.d("TEST", "Counter berubah : $count")  
        }) {  
            Text("Tambah")  
        }  
    }  
}
```

- Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawab :



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawab : Sama seperti sebelumnya, log "Counter berubah : <nilai>" tetap muncul setiap klik, karena baris Log.d dijalankan setiap kali event onClick terpicu.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab : Kali ini tampilan ikut berubah: teks "Counter : " pada layar bertambah sesuai nilai count yang sebenarnya ("Counter : 1", "Counter : 2", dan seterusnya) setiap kali tombol ditekan.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Jawab : Karena count sekarang dideklarasikan sebagai state Compose lewat `remember { mutableStateOf(0) }`. State ini bersifat observable — setiap perubahan nilainya diketahui oleh Compose runtime, sehingga Compose otomatis menjalankan recomposition pada bagian UI yang membaca count, dan tampilan ikut diperbarui. Pada percobaan sebelumnya count hanyalah variabel biasa yang tidak diamati Compose.

12. Apa yang dimaksud recomposition pada Compose?

Jawab : Recomposition adalah proses Compose menjalankan ulang fungsi composable (atau bagian composable) yang bergantung pada suatu state, setiap kali state tersebut berubah nilainya, agar tampilan yang dihasilkan selalu konsisten dengan data/state terbaru. Compose berusaha melakukan recomposition seefisien mungkin, hanya pada bagian UI yang benar-benar terpengaruh oleh perubahan state, bukan menggambar ulang seluruh layar.

13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:

```

fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    var text by remember { mutableStateOf("") }

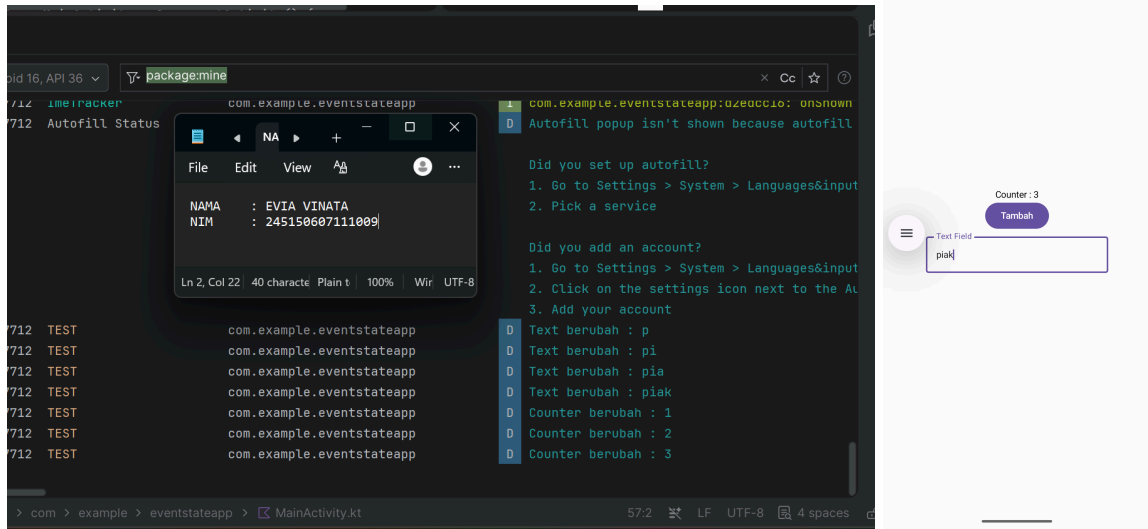
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()

    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
        OutlinedTextField(
            value = text,
            onChange = {
                text = it
                Log.d("TEST", "Text berubah : $it")
            },
            label = { Text("Text Field") }
        )
    }
}

```

14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawab :



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawab : Setiap huruf yang diketik/dihapus memunculkan log baru, misalnya "Text berubah : a", "Text berubah : ab", dst., karena callback `onValueChange` dipanggil setiap kali nilai `TextField` berubah (per karakter).

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawab : Karakter yang diketik langsung muncul dan diperbarui secara real-time di dalam kotak `OutlinedTextField` pada layar, mengikuti apa yang diketik pengguna.

17. Apa yang menyebabkan hal tersebut terjadi?

Jawab : Nilai `OutlinedTextField` diikat ke state `text` lewat `value = text`. Setiap kali pengguna mengetik, `onValueChange` mengubah state `text`; karena `text` adalah state `Compose`, perubahan ini memicu `recomposition` sehingga `OutlinedTextField` ditampilkan ulang dengan nilai `text` terbaru.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Jawab : Ada dua state: `count` (`Int`, untuk counter) dan `text` (`String`, untuk isi `TextField`), keduanya dideklarasikan dengan `remember { mutableStateOf(...) }`.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Jawab : Ada dua event: `onClick` pada Button "Tambah", dan `onValueChange` pada `OutlinedTextField`

20. Kapan event - event tersebut dijalankan?

Jawab : `onClick` dijalankan setiap kali pengguna menekan/mengetuk tombol "Tambah". `onValueChange` dijalankan setiap kali nilai teks pada `OutlinedTextField` berubah, yaitu setiap pengguna mengetik atau menghapus satu karakter di dalamnya.

Implisit Intent

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity

@Composable

```
fun ImplicitIntentTest(context: Context) {
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        // Buka Browser
        Button(onClick = {
            val intent = Intent(Intent.ACTION_VIEW).apply {
                data = Uri.parse("https://www.google.com")
            }
            context.startActivity(intent)
        }) {
            Text("Buka Browser")
        }
        Spacer(modifier = Modifier.height(16.dp))
        // Buka Dial Telepon
        Button(onClick = {
            val intent = Intent(Intent.ACTION_DIAL).apply {
                data = Uri.parse("tel:08123456789")
            }
            context.startActivity(intent)
        }) {
            Text("Buka Telepon")
        }
        Spacer(modifier = Modifier.height(16.dp))
        // Kirim Email (dengan chooser)
        Button(onClick = {
            val intent = Intent(Intent.ACTION_SENDTO).apply {
                data = Uri.parse("mailto:test@gmail.com")
            }
            val chooser = Intent.createChooser(intent, "Pilih aplikasi")
        }) {
            Text("Kirim Email")
        }
    }
}
```

```

email")
        context.startActivity(chooser)
    }) {
        Text("Kirim Email")
    }
    Spacer(modifier = Modifier.height(16.dp))
    // Buka Maps
    Button(onClick = {
        val intent = Intent(Intent.ACTION_VIEW).apply {
            data = Uri.parse("geo:-7.250445,112.768845")
        }
        context.startActivity(intent)
    }) {
        Text("Buka Maps")
    }
}
}

```

3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```

import android.content.Context
import android.content.Intent
import android.net.Uri
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

```

4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

```

setContent {
    val context = LocalContext.current
    ImplicitIntentTest(context)
}

```

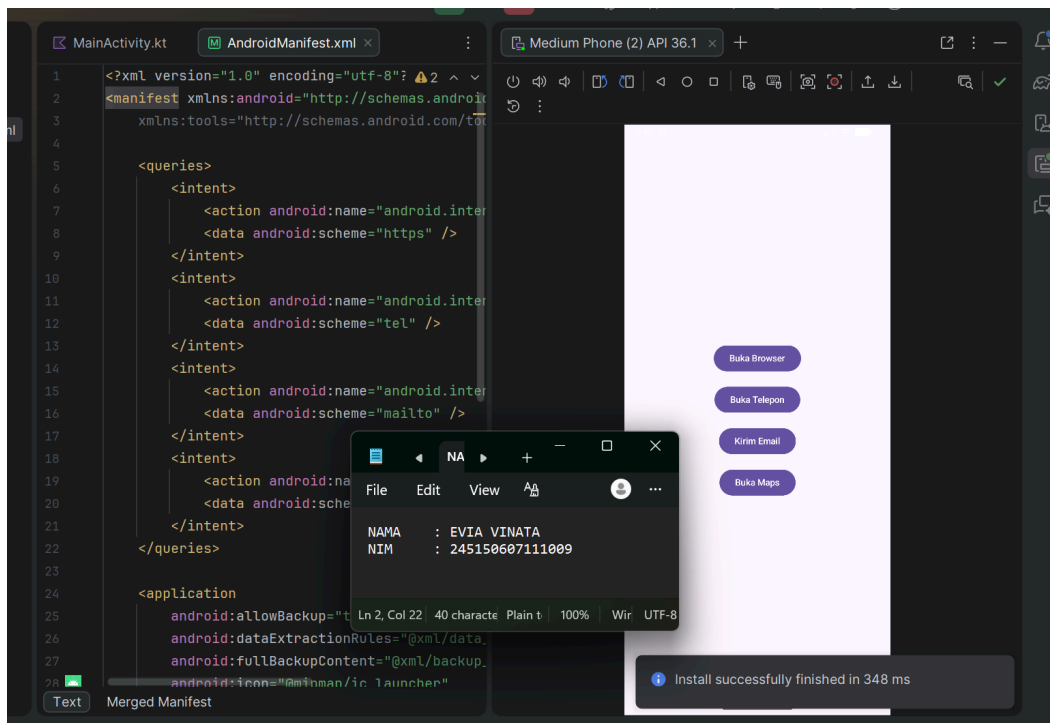
Aktivitas Percobaan

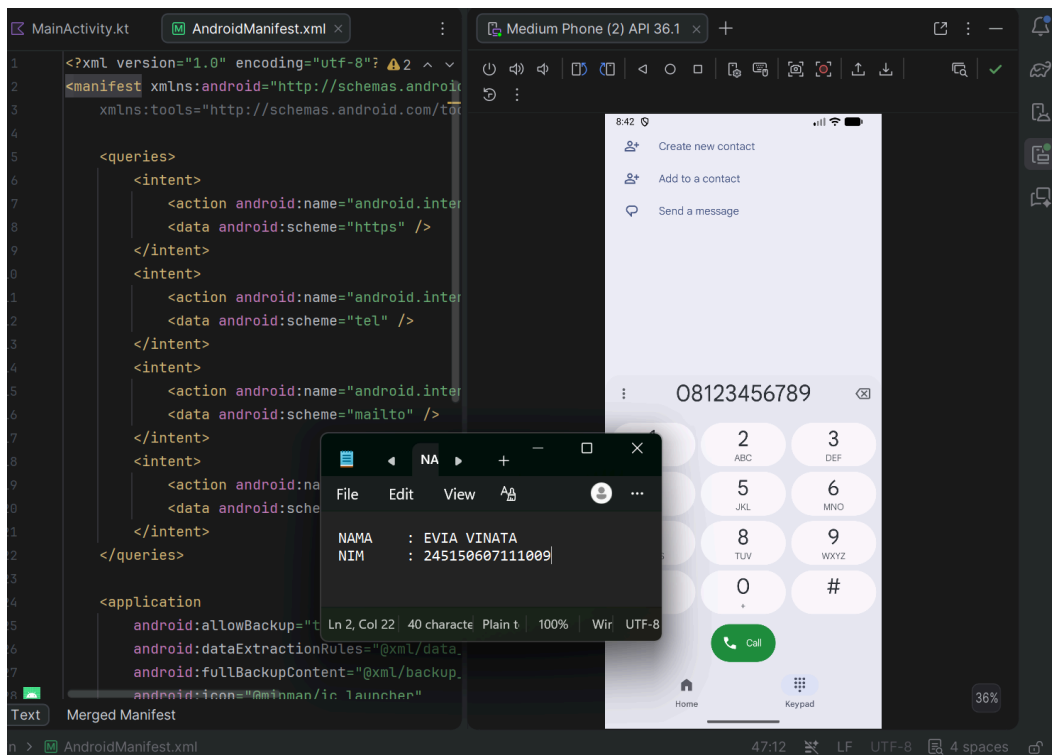
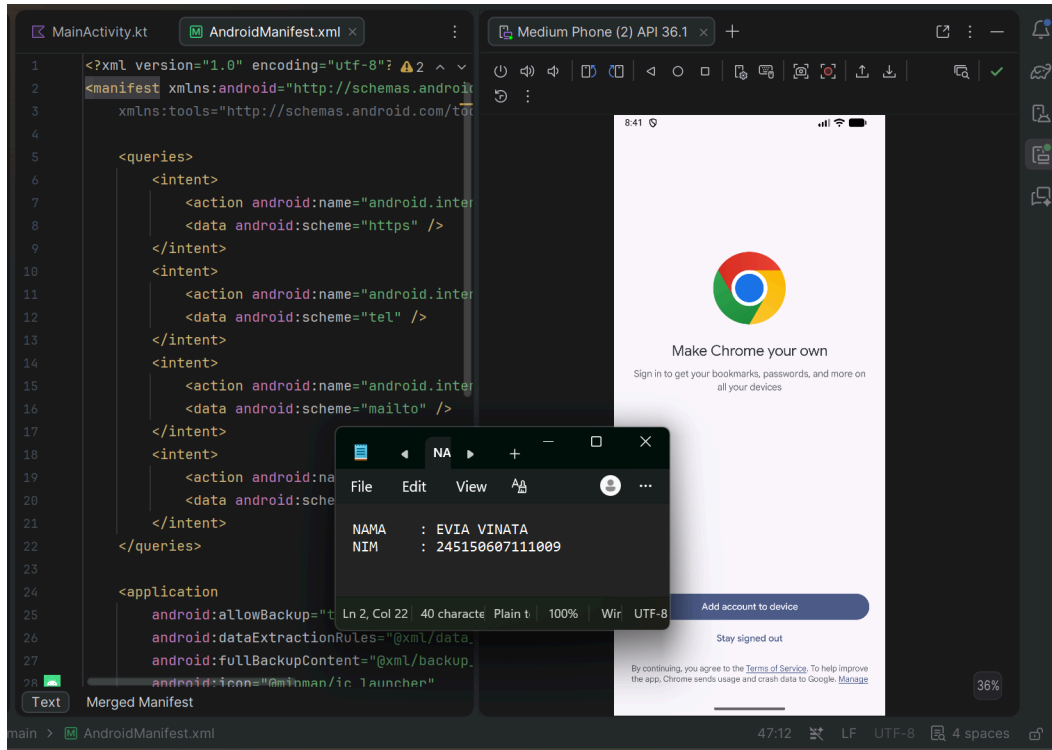
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:

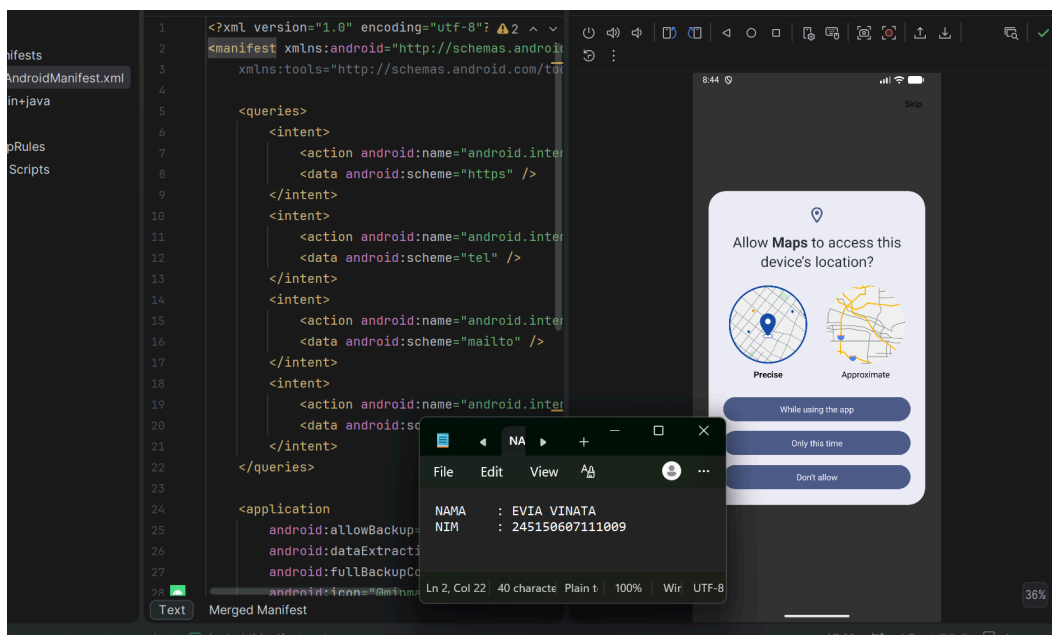
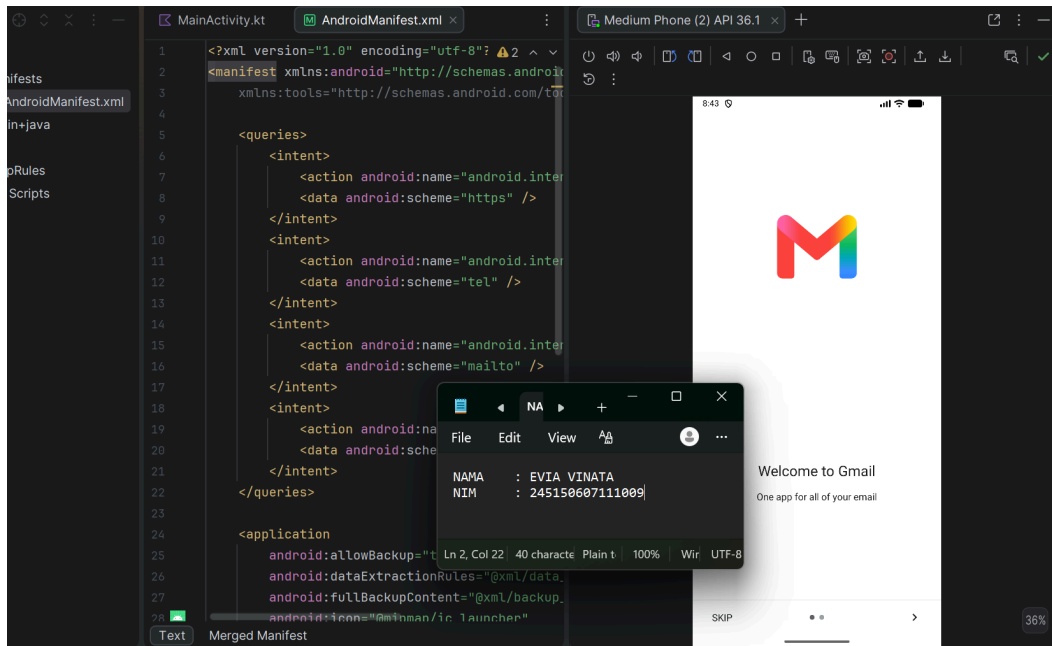
Jawab : Buka Browser: sistem membuka aplikasi browser (mis. Chrome) menuju alamat <https://www.google.com>. Buka Telepon: sistem membuka aplikasi dialer/telepon dengan nomor 08123456789 sudah terisi otomatis, namun panggilan belum otomatis dilakukan (pengguna harus menekan tombol panggil sendiri). Kirim Email: sistem menampilkan chooser aplikasi email yang tersedia (mis. Gmail) dengan alamat penerima test@gmail.com sudah terisi. Buka Maps: sistem membuka aplikasi peta (mis. Google Maps) menuju koordinat -7.250445, 112.768845.

2. Klik Buka Browser
3. Klik Buka Telepon
4. Klik Kirim Email
5. Klik Buka Maps
6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

Jawab :







7. Pada kode yang digunakan, terdapat beberapa action seperti:
8. Intent.ACTION_VIEW
9. Intent.ACTION_DIAL
10. Intent.ACTION_SENDTO
11. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Jawab : ACTION_VIEW adalah aksi umum untuk menampilkan/membuka suatu data kepada pengguna; sistem akan membuka aplikasi yang paling sesuai berdasarkan skema data yang diberikan (URL browser, geo: aplikasi peta). ACTION_DIAL secara khusus membuka aplikasi telepon/dialer dengan nomor yang sudah terisi, tetapi tidak langsung melakukan panggilan — ini lebih aman dibanding ACTION_CALL karena

masih memerlukan konfirmasi pengguna. ACTION_SENDTO digunakan untuk mengirim data ke tujuan tertentu berdasarkan skema (misalnya mailto: untuk email, smsto: untuk SMS), membuka aplikasi yang sesuai dengan konten yang sudah disiapkan (mis. alamat penerima).

12. Pada kode yang digunakan, terdapat beberapa data seperti:

13. data = Uri.parse("mailto:test@gmail.com")

14. data = Uri.parse("<https://www.google.com>")

15. data = Uri.parse("geo:-7.250445,112.768845")

16. Apa fungsi data pada intent ?

Jawab : Properti data (berupa URI) menentukan objek atau konten spesifik yang akan diproses oleh action, misalnya nomor telepon (tel:), alamat halaman web (https:), alamat email (mailto:), atau koordinat lokasi (geo:). Data inilah yang memberitahu sistem "apa" yang ingin diproses, sedangkan action memberitahu "aksi apa" yang ingin dilakukan terhadap data tersebut.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Jawab : Android mencocokkan action (jenis operasi) dan skema/tipe data pada Intent dengan intent-filter yang didaftarkan oleh aplikasi-aplikasi yang terpasang di perangkat. Kombinasi keduanya yang menentukan aplikasi mana saja yang dianggap "mampu" menangani permintaan tersebut. Jika hanya ada satu aplikasi yang cocok, sistem langsung membukanya; jika ada lebih dari satu aplikasi yang cocok, sistem menampilkan daftar pilihan (chooser) agar pengguna memilih sendiri aplikasi mana yang digunakan.

Referensi

Google (2024) The Activity Lifecycle. Available at:

<https://developer.android.com/guide/components/activities/activity-lifecycle>

Google (2024) Jetpack Compose layouts. Available at:

<https://developer.android.com/jetpack/compose/layouts>

Google (2024) Compose Modifiers. Available at:

<https://developer.android.com/jetpack/compose/modifiers>

Google Developers (2026) Thinking in Compose. Available at:

<https://developer.android.com/develop/ui/compose/mental-model> (Accessed: 8 September 2026).

Google Developers (2026) State and Jetpack Compose. Available at:

<https://developer.android.com/develop/ui/compose/state> (Accessed: 8 September 2026).

Glosarium

Event

Aksi atau kejadian yang dipicu oleh interaksi pengguna atau sistem, seperti klik tombol, input teks, atau perubahan data, yang kemudian akan memicu perubahan pada aplikasi.

Delegation (Property Delegation)

Mekanisme di Kotlin untuk mendelegasikan pengelolaan suatu properti ke objek lain, biasanya menggunakan kata kunci `by`.

Intent

Objek pada Android yang digunakan untuk melakukan komunikasi antar komponen aplikasi atau dengan aplikasi lain untuk menjalankan suatu aksi tertentu.

Implicit Intent

Intent yang tidak menyebutkan secara langsung komponen tujuan, melainkan hanya mendefinisikan aksi dan data sehingga sistem Android akan memilih aplikasi yang sesuai.

Imperatif

Pendekatan pemrograman UI yang memperbarui komponen tampilan secara langsung melalui referensi objek dan perubahan propertinya.

Deklaratif

Pendekatan pemrograman UI yang mendeskripsikan tampilan berdasarkan state saat ini; ketika state berubah, framework memperbarui UI yang diperlukan.

State

Data atau nilai yang merepresentasikan kondisi saat ini dari UI.

Dalam Jetpack Compose, perubahan state akan secara otomatis menyebabkan UI diperbarui (recomposition).